Recreating the Techasoft Scrolling Effects

Implementation guide for techasoft.com-style scroll animations, using GSAP, ScrollTrigger and Lenis

1. Read this first

I could read the rendered content and layout of https://www.techasoft.com/ but not its CSS or JavaScript files. This guide therefore **reconstructs** the effects from the page structure. It is not a copy of their code. The structure shows:

- A hero with four stacked messages (Application Development, Analytical Reporting, Digital Marketing, Software Robustness) over a poster/video background
- Ten numbered service cards (01 to 10)
- An industries list with ten items
- Two very long logo lists in which every logo appears twice, the usual sign of an infinite marquee
- A technology icon grid and a technology logo grid
- A contact section labelled "Scroll to begin"
- A testimonial slider
- A video slider with a "01 / 09" counter and a "Drag to rotate the cube" 3D cube
- Milestone counters that show "0+" in the static HTML and are filled in by JavaScript
- Blog cards and a chat widget

The exact timings, easing and trigger points are my best equivalents. Section 12 shows how to read the real values from their site so you can match them precisely.

Use your own images, text and code. Re-implementing a technique is fine, but do not copy their assets, copy or custom scripts.

2. Effects and the tools used

#	Effect	Where on the page	Technique
1	Smooth (inertial) scrolling	Whole page	Lenis
2	Pinned hero with changing messages	Top	GSAP ScrollTrigger (pin, scrub)
3	Fade/slide-up reveal	Most sections	ScrollTrigger batch

#	Effect	Where on the page	Technique
4	Stacking numbered cards	Services 01 to 10	CSS sticky and ScrollTrigger scale
5	Infinite logo marquee	Clients	CSS keyframes, duplicated track
6	Count-up numbers	Milestones	ScrollTrigger once and a GSAP tween
7	Draggable 3D cube	Video section	CSS 3D transforms and pointer events
8	Slider with counter	Videos, testimonials	Swiper (or any slider)

3. Project setup

Create this structure:

```
project/
  index.html
  css/styles.css
  js/main.js
```

Add these tags to `index.html`. Put the scripts just before `</body>`, in this order:

```
<meta name="viewport" content="width=device-width, initial-scale=1">
<link rel="stylesheet" href="css/styles.css">

<script src="https://cdn.jsdelivr.net/npm/gsap@3.12.5/dist/gsap.min.js">
</script>
<script
src="https://cdn.jsdelivr.net/npm/gsap@3.12.5/dist/ScrollTrigger.min.js">
</script>
<script src="https://cdn.jsdelivr.net/npm/lenis@1.1.13/dist/lenis.min.js">
</script>
<script src="js/main.js"></script>
```

Start `js/main.js` with the setup that every later section builds on:

```
gsap.registerPlugin(ScrollTrigger);

const reduceMotion = window.matchMedia('(prefers-reduced-motion:
reduce)').matches;
```

In plain CSS, add:

```
:root { --ease: cubic-bezier(.22,.61,.36,1); }
html.lenis, html.lenis body { height: auto; }
.lenis.lenis-smooth { scroll-behavior: auto !important; }
body { overflow-x: hidden; }
```

4. Effect 1: Smooth scrolling (Lenis)

This is the "buttery" feel where the page glides to a stop instead of jumping. Lenis must be wired into ScrollTrigger so that pinned and scrubbed animations stay in sync.

```
let lenis;
if (!reduceMotion) {
  lenis = new Lenis({
    duration: 1.2,
    easing: t => Math.min(1, 1.001 - Math.pow(2, -10 * t)),
    smoothWheel: true
  });

  lenis.on('scroll', ScrollTrigger.update);
  gsap.ticker.add(time => lenis.raf(time * 1000));
  gsap.ticker.lagSmoothing(0);
}
```

Tips:

- Lower `duration` (for example 0.8) feels snappier. Higher (1.5) feels floatier.
- For in-page links, use `lenis.scrollTo('#contact', { offset: -80 })`.
- Never combine Lenis with CSS `scroll-behavior: smooth`. The two fight each other.

5. Effect 2: Pinned hero with changing messages

The hero stays fixed while you scroll. Each of the four messages fades in, holds, then fades out as scroll progress advances.

HTML

```
<section class="hero">
  <video class="hero__bg" autoplay muted loop playsinline poster="img/hero-
poster.jpg">
    <source src="video/hero.mp4" type="video/mp4">
  </video>
  <div class="hero__intro">
    <h1>Enterprise AI &amp; Software Development Company</h1>
    <p>Build smarter, faster, and scalable solutions.</p>
  </div>
  <div class="hero__slides">
    <article class="slide"><h2>Application Development</h2><p>Full-cycle
software development...</p></article>
    <article class="slide"><h2>Analytical Reporting</h2><p>Accurate,
consistent reporting...</p></article>
    <article class="slide"><h2>Digital Marketing</h2><p>Reach your audience
better...</p></article>
    <article class="slide"><h2>Software Robustness</h2><p>Black-box and
white-box testing...</p></article>
  </div>
</section>
```

CSS

```
.hero { position: relative; height: 100vh; overflow: hidden; color: #fff; }
.hero__bg { position: absolute; inset: 0; width: 100%; height: 100%; object-
fit: cover; }
.hero__intro, .slide {
  position: absolute; inset: 0;
  display: flex; flex-direction: column; justify-content: center;
  padding: 0 8vw;
}
.slide { opacity: 0; }
```

JavaScript

```
if (!reduceMotion) {
  const slides = gsap.utils.toArray('.slide');

  const tl = gsap.timeline({
    scrollTrigger: {
      trigger: '.hero',
      start: 'top top',
      end: () => '+=' + window.innerHeight * (slides.length + 1),
      pin: true,
      scrub: 0.6,
      anticipatePin: 1
```

```
    }
  });

  tl.to('.hero__intro', { opacity: 0, y: -40, duration: 0.5 });

  slides.forEach(slide => {
    tl.fromTo(slide, { opacity: 0, y: 60 }, { opacity: 1, y: 0, duration: 0.5
})
      .to(slide, { opacity: 0, y: -60, duration: 0.5 }, '+=0.6');
  });
}
```

How it works:

- `pin: true` holds the hero in place for the scroll distance set by `end`. Here that is one screen height per message.
- `scrub: 0.6` ties the animation to scroll position with a 0.6 second smoothing lag.
- The `'+=0.6'` gap holds each message on screen before it fades out.

6. Effect 3: Fade and slide-up reveals

Sections, headings and cards ease in as they enter the viewport.

CSS: mark anything you want revealed with a class.

```
.reveal { opacity: 0; transform: translateY(40px); }
```

JavaScript

```
if (reduceMotion) {
  gsap.set('.reveal', { opacity: 1, y: 0 });
} else {
  ScrollTrigger.batch('.reveal', {
    start: 'top 88%',
    once: true,
    onEnter: batch => gsap.to(batch, {
      opacity: 1, y: 0,
      duration: 0.9,
      ease: 'power3.out',
      stagger: 0.12,
      overwrite: true
    })
  });
}
```

`ScrollTrigger.batch` groups elements that enter at the same time, which gives the staggered cascade you see on card grids such as Industries, Technologies and Blogs.

7. Effect 4: Stacking numbered service cards

Cards 01 to 10 slide up and pile on top of each other. Each earlier card shrinks slightly as the next one covers it.

HTML

```
<section class="services">
  <h2>Our Services</h2>
  <div class="stack">
    <article class="card"><span>01</span><h3>Artificial Intelligence</h3>
<p>...</p></article>
    <article class="card"><span>02</span><h3>Machine Learning</h3><p>...</p>
</article>
    <!-- repeat through 10 -->
  </div>
</section>
```

CSS

```
.card {
  position: sticky;
  top: calc(90px + var(--i) * 18px);
  min-height: 60vh;
  margin-bottom: 8vh;
  padding: 3rem;
  border-radius: 24px;
  background: #fff;
  box-shadow: 0 20px 60px rgba(0,0,0,.12);
  transform-origin: center top;
}
```

JavaScript

```
const cards = gsap.utils.toArray('.card');
cards.forEach((card, i) => {
  card.style.setProperty('--i', i);

  if (!reduceMotion && i < cards.length - 1) {
    gsap.to(card, {
      scale: 0.94,
      opacity: 0.7,
      ease: 'none',
      scrollTrigger: {
        trigger: cards[i + 1],
        start: 'top 85%',
        end: 'top 25%',
        scrub: true
      }
    });
  }
});
```

Important: `position: sticky` stops working if any ancestor has `overflow: hidden`, `overflow: auto` or `overflow: clip`. Check the cards' parent containers if the stacking does not happen.

8. Effect 5: Infinite logo marquee (Clients)

The original lists every client logo twice. That duplication is what makes the loop seamless: the track slides left by exactly half its width, then restarts without a visible jump.

HTML

```
<div class="marquee">
  <div class="marquee__track">
    <!-- your logos, first set -->
    <img src="logos/a.svg" alt="Client A"> <img src="logos/b.svg" alt="Client
B">
    <!-- the SAME logos again, second set (use aria-hidden on this copy) -->
    <img src="logos/a.svg" alt="" aria-hidden="true"> <img src="logos/b.svg"
alt="" aria-hidden="true">
  </div>
</div>
```

The site appears to use two or more rows. To copy that, add a second `.marquee` with the class `marquee--reverse`.

CSS

```
.marquee { overflow: hidden; }
.marquee__track {
  display: flex;
  gap: 3rem;
  width: max-content;
  animation: scroll-left 40s linear infinite;
}
.marquee--reverse .marquee__track { animation-direction: reverse; }
.marquee:hover .marquee__track { animation-play-state: paused; }
.marquee img { height: 56px; width: auto; flex: none; }

@keyframes scroll-left {
  from { transform: translateX(0); }
  to   { transform: translateX(-50%); }
}

@media (prefers-reduced-motion: reduce) {
  .marquee__track { animation: none; }
}
```

The loop only works when both sets are identical and the gap is applied evenly. If you see a jump, check that the two sets have the same number of items.

9. Effect 6: Count-up milestone numbers

The static HTML shows `0+`. JavaScript counts up to the real value when the section scrolls into view.

HTML

```
<div class="milestone"><span class="count" data-target="15">0</span>+
<small>Years Experience</small></div>
<div class="milestone"><span class="count" data-target="500">0</span>+
<small>Apps, Software &amp;amp; Websites Built</small></div>
```

JavaScript

```
gsap.utils.toArray('.count').forEach(el => {
  const target = Number(el.dataset.target);

  if (reduceMotion) { el.textContent = target; return; }

  const counter = { val: 0 };
  ScrollTrigger.create({
    trigger: el,
    start: 'top 85%',
    once: true,
    onEnter: () => gsap.to(counter, {
      val: target,
      duration: 2,
      ease: 'power2.out',
      onUpdate: () => { el.textContent =
Math.round(counter.val).toLocaleString(); }
    })
  });
});
```

Replace the numbers with your own real figures.

10. Effect 7: Draggable 3D cube

The video section has a cube with four faces and the hint "Drag to rotate the cube".

HTML

```
<div class="scene">
  <div class="cube" id="cube">
    <div class="face front"><img src="img/front.jpg" alt=""></div>
    <div class="face back"><img src="img/back.jpg" alt=""></div>
    <div class="face right"><img src="img/right.jpg" alt=""></div>
    <div class="face left"><img src="img/left.jpg" alt=""></div>
  </div>
  <p>Drag to rotate the cube</p>
</div>
```

CSS

```
.scene { width: 280px; height: 280px; perspective: 900px; margin: auto; }
.cube {
  position: relative; width: 100%; height: 100%;
  transform-style: preserve-3d;
  cursor: grab; touch-action: none;
}
.face { position: absolute; inset: 0; backface-visibility: hidden; }
.face img { width: 100%; height: 100%; object-fit: cover; }
.front { transform: rotateY(0deg)   translateZ(140px); }
.right { transform: rotateY(90deg)  translateZ(140px); }
.back  { transform: rotateY(180deg) translateZ(140px); }
.left  { transform: rotateY(-90deg) translateZ(140px); }
```

The `translateZ` value must be half of the cube's width (140px for a 280px cube).

JavaScript

```
const cube = document.getElementById('cube');
let rotY = -20, rotX = -10, dragging = false, lastX = 0, lastY = 0;

const render = () => {
  cube.style.transform = `rotateX(${rotX}deg) rotateY(${rotY}deg)`;
};
render();

cube.addEventListener('pointerdown', e => {
  dragging = true; lastX = e.clientX; lastY = e.clientY;
  cube.setPointerCapture(e.pointerId);
  cube.style.cursor = 'grabbing';
});
cube.addEventListener('pointermove', e => {
  if (!dragging) return;
  rotY += (e.clientX - lastX) * 0.5;
  rotX -= (e.clientY - lastY) * 0.5;
  lastX = e.clientX; lastY = e.clientY;
  render();
});
const stop = () => { dragging = false; cube.style.cursor = 'grab'; };
cube.addEventListener('pointerup', stop);
cube.addEventListener('pointercancel', stop);
```

Optional: add a slow auto-rotate with `requestAnimationFrame` that pauses while the user is dragging.

11. Effect 8: Sliders with a counter, and the "Scroll to begin" section

Video and testimonial sliders. The "01 / 09" counter is a standard slider pagination. The easiest way to match it is Swiper:

```
<link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/swiper@11.1.14/swiper-bundle.min.css">
<script src="https://cdn.jsdelivr.net/npm/swiper@11.1.14/swiper-
bundle.min.js"></script>

<div class="swiper videos">
  <div class="swiper-wrapper">
    <div class="swiper-slide">...</div>
    <div class="swiper-slide">...</div>
  </div>
  <div class="counter"><span id="cur">01</span> / <span id="tot">09</span>
</div>
</div>
```

```
const pad = n => String(n).padStart(2, '0');
const swiper = new Swiper('.videos', {
  speed: 700,
  grabCursor: true,
  on: {
    init(s) { document.getElementById('tot').textContent =
pad(s.slides.length); },
    slideChange(s) { document.getElementById('cur').textContent =
pad(s.realIndex + 1); }
  }
});
```

"Scroll to begin" contact section. This is a section that pins, shows the prompt, and then reveals the form as you scroll. A simple version:

```
if (!reduceMotion) {
  gsap.timeline({
    scrollTrigger: {
      trigger: '.contact',
      start: 'top top',
      end: '+=100%',
      pin: true,
      scrub: 0.5
    }
  })
  .to('.contact__prompt', { opacity: 0, y: -30 })
  .fromTo('.contact__form', { opacity: 0, y: 60 }, { opacity: 1, y: 0 },
'<0.2');
}
```

12. Verify against the original site (to match it exactly)

Because I could not read their scripts, use your browser to confirm the real settings:

1. Open https://www.techasoft.com/ in Chrome, press F12 and open the **Network** tab. Filter by **JS** and reload. Script file names usually reveal the libraries (for example gsap, ScrollTrigger, lenis, swiper, AOS).
2. In the **Console**, type `typeof gsap`, `typeof ScrollTrigger`, `typeof Lenis` and `typeof AOS`. A result other than `"undefined"` confirms that library is in use. If `ScrollTrigger` exists, `ScrollTrigger.getAll()` lists every trigger with its start, end and pin settings.
3. In **Elements**, select a card or the hero and look at the **Computed** and **Styles** panels. Look for `position: sticky`, `will-change: transform` and `animation: ...` rules.
4. Use the **Animations** panel (three-dot menu, More tools, Animations) to see CSS animation durations and easing, such as the marquee.
5. Use **Performance** recording while scrolling if you want to compare smoothness.

Copy the numbers you find (durations, offsets, easing names) into the code above.

13. Performance and accessibility

- Animate only `transform` and `opacity`. Avoid animating `top`, `left`, `width` or `height`.
- Compress the hero video (aim for under 3 MB), keep a `poster` image, and add `preload="metadata"`.
- Lazy-load below-the-fold images with `loading="lazy"`.

- Honour `prefers-reduced-motion`. The code above already skips pins, reveals and Lenis for those users.
- Call `ScrollTrigger.refresh()` after images or fonts finish loading, because layout shifts move the trigger positions:

```
window.addEventListener('load', () => ScrollTrigger.refresh());
```

- On mobile, test pinned sections carefully. Consider shorter pin distances or turning pinning off below 768px:

```
const mm = gsap.matchMedia();
mm.add('(min-width: 768px)', () => {
  // put the hero pin timeline and the stacking cards here
});
```

- Do not lock keyboard scrolling. Lenis keeps normal keyboard, scrollbar and anchor behaviour.

14. Build checklist

1. Add the GSAP, ScrollTrigger and Lenis scripts and wire Lenis to the GSAP ticker.
2. Build the hero HTML and CSS, then add the pinned timeline.
3. Add `.reveal` to sections and cards, and add the batch animation.
4. Build the services stack with `position: sticky` and the scale-down tween.
5. Add the duplicated-track marquee for client logos.
6. Add the counters with `data-target` values.
7. Build the cube, then the sliders.
8. Add the `prefers-reduced-motion` and mobile fallbacks.
9. Run `ScrollTrigger.refresh()` on load and test at 1440px, 1024px and 390px widths.
10. Compare against the original in DevTools (Section 12) and tune durations and offsets.

If you tell me your stack (plain HTML, React, Next.js, WordPress) I can adapt these snippets to it, for example turning them into React components with `useGSAP`.